

cloudfire / develop — Step 4 outline

A simple MLP neural field with Fourier-feature encoding that predicts the per-pixel parameters as a smooth function of *space and time*, trained *through* the step-3 physics (Option B from step 3). Deliverable of `instructions.md` step 4. Validated on a small instance locally; the full cube is a server run (step 5).

1. OBJECTIVE

Replace step-3's *free per-pixel* parameter tensors with the outputs of a coordinate network $f_\theta(x, y, t)$. The physics core is untouched: f_θ predicts the parameters, which flow through `composite_synth` into `chi2_per_pixel`. Optimising the *network weights* θ (not per-pixel values) supplies the regularisation — **smooth in space and time**, because one set of weights serves all (x, y, t) and the Fourier bandwidth caps roughness. This breaks the step-3 background/cloud degeneracy and couples frames in time.

2. ARCHITECTURE (SIMPLE)

Normalise coordinates to $\mathbf{v} = (x, y, t) \in [-1, 1]^3$. **Fourier features** (fixed, Gaussian random, *anisotropic*):

$$\gamma(\mathbf{v}) = [\sin(2\pi\mathbf{B}\mathbf{v}), \cos(2\pi\mathbf{B}\mathbf{v})], \quad \mathbf{B} \in \mathbb{R}^{m \times 3},$$

with the two space columns of $\mathbf{B}$ scaled by σ_{xy} and the time column by σ_t — independent control of spatial vs temporal smoothness. **MLP:** $\gamma \rightarrow$ a few hidden layers ($W \sim 128\text{--}256$, $D \sim 4$, SiLU) $\rightarrow \mathbb{R}^{14}$. **Affine output head** anchored at the step-3 inits, $\mathbf{p} = \mathbf{s} \odot \text{MLP}(\gamma(\mathbf{v})) + \mathbf{p}_0$ with small $\mathbf{s}$, so at init the field predicts the good step-3 guess everywhere and learns deviations.

3. OUTPUT $\rightarrow$ PHYSICS $\rightarrow$ LOSS (REUSED)

$\log a$ is **frozen** (per your note): the field outputs **14** numbers — `c`(6) and 8 cloud values ($S, \Delta\tau, v_{\text{los}}, \Delta v$ per cloud). We *insert* the fixed $\log a = (-4, -5)$ to assemble `p_cloud`(10), then call `composite_synth` and `chi2_per_pixel` *unchanged* (reuse `init_cloud_params`'s column layout). Positivity via the `abs` already in the cloud model. Loss = mean per-pixel χ^2 over the sampled (x, y, t) ; Adam on θ .

4. THE KNOBS: TWO BANDWIDTHS = SPACE & TIME SMOOTHNESS

σ_{xy} and σ_t are the regulariser: they set the spatial and temporal frequencies the maps may contain — small $\Rightarrow$ smoother/more-regularised, large $\Rightarrow$ finer detail (toward step-3 per-pixel freedom). They replace `utils.regu2` (space) and any explicit temporal penalty. Scan a few of each; optional weak weight-decay on θ as a secondary prior.

5. TEST LOCALLY BEFORE THE SERVER

The full 4-D fit is a server job, but the code is validated first on a *tiny instance* that runs in seconds on the laptop CPU:

- **Shrink via config:** small spatial crop (e.g. 24×24), a few frames (3–5 t around 13), narrow/shallow MLP, few iterations, `device=cpu`.
- This exercises the *entire* path (Fourier $\rightarrow$ MLP $\rightarrow$ `composite_synth` $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ step): shapes, gradients, NaN-safety, and that the loss decreases. If the tiny run trains, the code is correct.
- **Config-driven scale:** crop, frame set, W, D, m , batch size, iters and device are top-of-notebook knobs, so the *identical* code becomes the full run by enlarging them and setting `device=cuda` — no code change to deploy.
- **Minibatch over coordinates:** the field is evaluated per (x, y, t) sample, so memory is bounded by the batch, not the cube; big problems run in minibatches (slower locally, full-size on the server).
- Use the tiny run's measured per-iter time and memory to *size the server* (scale by $\#\text{samples} \times \text{iters}$).

6. MODULE & NOTEBOOK (AFTER APPROVAL)

`pinn_model.py`: `FourierFeatures` (fixed anisotropic $\mathbf{B}$) and `ParameterField` (`forward(coords)  $\rightarrow$  c(N, 6), p_cloud(N, 10) with frozen  $\log a$ ), reusing composite_model. A step-4 notebook defaulting to the laptop-sized smoke test, comparing against step 3 on the small instance (similar  $\chi^2$ , smoother space-time maps).`

Plan only — no code until you approve. Then `pinn_model.py` + a step-4 notebook (tiny-instance default). Step 5 = the full-cube run on the server: enlarge the config, `device=cuda`, same code.